

Free Claude Code Plugin Brings Amazon's PR/FAQ Process to Engineers and Founders

punt-labs releases `prfaq`, an open-source tool to generate, review, debate, and iterate on product discovery documents inside the terminal

Press Release

SAN FRANCISCO — September 2026 — Punt Labs released `prfaq`, a free Claude Code plugin that guides engineers and founders through Amazon's Working Backwards PR/FAQ process to evaluate product ideas before building them. The plugin provides seven commands spanning the full product-thinking workflow: `/prfaq` generates a document from a structured discovery conversation, `/prfaq:review` runs automated peer review against Working Backwards principles, `/prfaq:research` finds and cites evidence for unsupported claims, `/prfaq:feedback-to-us` evaluates the idea against Cagan's four risks framework, `/prfaq:meeting` simulates an Amazon-style review meeting with agentic personas, `/prfaq:meeting-hive` orchestrates that meeting as autonomous multi-agent debate, and `/prfaq:feedback` surgically redrafts the document based on pointed direction — all inside the same Claude Code session where the user already works (see [FAQ 14](#)). Engineers who ship products at unprecedented speed can now apply the same rigor to deciding *which* products to ship.

Problem

AI coding tools let a single person build in a weekend what used to take a team a quarter. The bottleneck has shifted: the hard part is no longer building — it is knowing what to build and why (see [FAQ 11](#)). Builders using Claude Code, Cursor, and similar tools ship fast, but most lack formal product training. They skip customer definition, competitive analysis, and unit economics — and discover after weeks of building that the product does not resonate.

The consequences are compounding. AI-generated code carries its own quality burden: 66% of developers cite “almost-right” AI output as their top frustration, and 45% report that debugging AI-generated code takes longer than expected [1]. The deeper problem is strategic: many products built during the current “vibe coding” wave [2] face significant rebuilds not because the code is bad, but because the code solves the wrong problem with confidence. Enterprises carry \$1.5–2 trillion in accumulated technical debt [3]; AI-accelerated development risks creating more of the same debt, faster.

A builder who wants product discipline has three options: hire a product manager they cannot afford, read a book and apply frameworks manually, or iterate — ship something, get feedback, ship again. Iteration works. But iteration has real costs: every cycle burns tokens, calendar time marketing to early users, and the founder's energy. Getting customer feedback is often the actual bottleneck. Builders would develop stronger product sense if the frameworks were accessible in the workflow where they already build, not in a book or workshop they will never attend.

Solution

`prfaq` brings product thinking into the builder's environment. Type `/prfaq` in any Claude Code session to start a structured conversation: Claude asks about the target customer, the problem, the competitive landscape, and the business model. It asks hard questions — the kind a product manager would ask before greenlighting a project.

Claude generates a complete PR/FAQ document: a mock press release describing the product as if it has already shipped, followed by external FAQs (what a customer would ask) and internal FAQs (what leadership, finance, and engineering would ask). The output is a LaTeX-compiled PDF (see [FAQ 6](#)): a clean, professional artifact for sharing with co-founders, investors, or advisors. Not a disposable brainstorm. A decision-making document designed to be read, debated, and revised.

Generation is step one. `/prfaq:review` runs automated peer review against Working Backwards principles and a cognitive bias checklist, surfacing weaknesses the author is too close to see. `/prfaq:research` finds evidence for unsupported claims — scanning local files, indexed documents, and the web — and embeds citations into the document. `/prfaq:feedback-to-us` evaluates the idea against Marty Cagan’s four risks framework [4] — value, usability, feasibility, and viability — producing an honest assessment of whether the product is worth building.

`/prfaq:meeting` simulates an Amazon-style PR/FAQ review meeting with agentic personas — a target customer, a principal engineer, a unit economics reviewer, a UX bar raiser, and a skeptical executive — each challenging the document from their perspective. `/prfaq:meeting-hive` takes this further: the personas debate autonomously, surfacing conflicts and consensus without the user moderating every exchange. The user is the PM and final decision-maker; the agents provide cross-functional scrutiny a solo builder otherwise lacks.

`/prfaq:feedback` takes pointed direction — “focus on CTOs, not solo devs” or “the TAM is too large” — and surgically redrafts affected sections, tracing cascading effects so the user iterates on substance rather than formatting. Together, the seven commands form a complete product-thinking workflow: generate, review, debate, and iterate.

Customer Quote

*“I kept building MVPs that nobody used. I’d spend three weeks coding, show it to people, and get polite shrugs. **prfaq** made me realize I was solving my own problem, not my customer’s. The PR/FAQ took forty-five minutes and saved me a month. The hardest question it asked was ‘what evidence do you have that customers want this?’ I didn’t have any. That was the answer.”*

— Illustrative composite based on builder community patterns

Getting Started

Install with one command — no account, no subscription, no configuration:

```
curl -fsSL https://raw.githubusercontent.com/punt-labs/prfaq/main/install.sh | bash
```

Then follow the workflow:

Generate. Type `/prfaq`. Claude asks discovery questions about the target customer, problem, differentiation, and business model, then drafts a complete PR/FAQ document. Within an hour, you have a compiled PDF.

Stress-test. Type `/prfaq:review` for automated peer review and `/prfaq:research` to find and cite evidence. Type `/prfaq:meeting` to face a simulated review meeting with agentic personas who challenge the document from every angle — or `/prfaq:meeting-hive` for autonomous multi-agent debate.

Iterate. Type `/prfaq:feedback` followed by a directional instruction. The plugin traces cascading effects and surgically redrafts affected sections. Repeat until the document — and the product idea — is ready.

Spokesperson Quote

“The people building products with AI make product decisions every day — they just don’t always have the frameworks. We built `prfaq` because the engineers and founders using Claude Code are also deciding what to build and for whom, and they deserve a tool that meets them where they work. The PR/FAQ process has been proven at Amazon for two decades. We didn’t invent it. We made it accessible inside a terminal.”

— **Jim Freeman**, Founder, Punt Labs

Call to Action

`prfaq` is free and open source, available now at <https://github.com/punt-labs/prfaq>. Install it, type `/prfaq`, and find out if your next product idea is worth building before you build it.

Frequently Asked Questions

External FAQs

Q1: What is prfaq and who is it for?

prfaq is a free Claude Code plugin that guides engineers and founders through the Amazon Working Backwards PR/FAQ process. The primary audience is technical builders who use AI coding tools to create products and want to validate an idea before committing weeks or months of development time. As Claude Code's user base expands beyond professional engineers [5], features like `/prfaq:import` — which enriches an existing document rather than generating one from scratch — extend reach to non-technical users who adopt Claude Code as a thinking partner.

Q2: How is this different from using a PR/FAQ template?

A template is a blank page with headings. prfaq is an interactive process: Claude asks discovery questions, challenges weak answers, identifies gaps in customer evidence, and generates the document from your responses. It applies the four risks framework to evaluate your idea honestly, not just format it. A template tells you *what* to write. prfaq helps you *think*.

Q3: How do I get started?

Run the one-line installer, then type `/prfaq` in any Claude Code session:

```
curl -fsSL https://raw.githubusercontent.com/punt-labs/prfaq/main/install.sh | bash
```

The plugin walks you through the process. No account, no API keys, no configuration. Installation takes under a minute.

Q4: What does the output look like?

A professionally typeset PDF compiled from LaTeX. The document includes a press release, external and internal FAQs, and a four-risks assessment. Designed to be shared with co-founders, investors, or advisors — not a throwaway brainstorm.

Q5: Do I need to know LaTeX?

No. The plugin generates the LaTeX source and compiles it automatically. You never see or edit LaTeX unless you choose to. You need a TeX distribution installed (TeX Live or similar); the installer checks for it and provides setup instructions if missing.

Q6: Why LaTeX instead of Markdown?

Three reasons. First, a PR/FAQ is a decision document shared with co-founders, investors, and advisors. LaTeX produces professionally typeset PDFs with consistent typography, precise layout, and visual credibility that Markdown-to-PDF pipelines do not match. The medium signals the thinking is serious. Second, LaTeX's structured environments (`faqpair`, `customerquote`, `spokespersonquote`) give the LLM a precise grammar to write into — the AI reads and writes `.tex`, the human reads the compiled `.pdf`. This separation means the source format can be rich and semantic without burdening the author. Third, LaTeX's package ecosystem (`biblatex` for citations, `mdframed` for callout boxes, `enumitem` for structured lists) provides extensibility Markdown lacks without custom renderers.

The trade-off is friction: LaTeX requires a TeX distribution (~4 GB), and users unfamiliar with it may find compilation errors intimidating. The installer mitigates this with automated setup, and the plugin handles all LaTeX generation — the user never writes `.tex` directly. This is a two-way door: if user feedback shows the TeX dependency is a meaningful adoption barrier, adding a Markdown output mode is straightforward since content generation is format-independent. The structured thinking matters more than the output format.

Q7: What happens to my data?

Everything runs locally. The plugin is a set of prompt files and a LaTeX template — it does not phone home, collect telemetry, or store data. Your PR/FAQ content is processed by Claude Code using your existing Anthropic API relationship. The generated `.tex` file and compiled PDF stay on your machine. Data handling is governed by Claude Code's privacy model and your Anthropic terms of service.

Q8: How much does it cost?

Free. `prfaq` is open source under a permissive license. No paid tier, no freemium upsell, no planned monetization. The plugin itself adds zero cost. You need an active Claude Code subscription or API access, which is a separate cost from Anthropic.

For API users, the token cost is small. A full PR/FAQ session — discovery conversation, document generation, and peer review — consumes roughly 100–200K input tokens and 20–40K output tokens across multiple turns (context is re-sent each turn). At Sonnet pricing (\$3/\$15 per million tokens, as of Q1 2026), that is \$0.60–1.20 per initial draft. Each feedback cycle is lighter: roughly 40–80K input tokens and 10–20K output tokens, or \$0.25–0.55 per round. A typical document through three or four revision cycles costs under \$4. Opus sessions cost roughly 5× more at list pricing.

For comparison, experienced product consultants charge \$100–300/hour. Claude Code subscribers on the Pro plan (\$20/month), Max 5× (\$100/month), or Max 20× (\$200/month) (as of Q1 2026) pay nothing incremental — token usage falls within the subscription's included allowance. Pricing changes; check <https://anthropic.com/pricing> for current rates.

Q9: Can I iterate on the document?

Yes. Type `/prfaq:feedback` with a directional instruction — “reframe the problem around compliance risk” or “the TAM is too small” — and the plugin traces cascading effects and surgically redrafts affected sections. Each feedback cycle recompiles the PDF and runs peer review automatically. You can also edit the `.tex` file directly or re-run the full discovery process. The Working Backwards process is designed for rapid iteration — the point is to iterate on a short document instead of a shipped product.

Internal FAQs

Value & Market

Q10: What is the total addressable market?

The addressable audience spans three segments: people building products with AI who lack product discipline, experienced product thinkers who want AI leverage to operate leaner, and the growing Claude Code user base that delivers both.

Segment 1: Non-technical builders who need product discipline. A massive wave of non-programmers is building software. Lovable has nearly 8 million users [6]. Replit has 35M+ users and its CEO pivoted away from professional developers [7]. Bolt reached \$40M ARR [8]. Cursor has 1M daily active users [9]. A quarter of startups in Y Combinator’s W25 cohort have codebases almost entirely AI-generated [10]. These builders ship fast but most lack formal product training — they skip customer definition, competitive analysis, and unit economics. The “vibe coding” wave [2] produces unprecedented build velocity with near-zero product discipline. This is the largest segment and the one with the most acute need for the frameworks prfaq provides.

Segment 2: Expert PMs seeking AI leverage. Experienced product managers and product-minded founders already know frameworks like Working Backwards [11] and four risks [4]. Their constraint is capacity, not knowledge. A PM at a startup wearing four hats does not have time to write a thorough PR/FAQ from scratch. An AI tool that drafts the document from a structured conversation, applies the framework rigorously, and handles formatting lets a single PM do work that previously required a PM plus an analyst plus a designer. This segment is smaller but higher-intent and more likely to produce documents worth sharing.

Segment 3: Claude Code crossing the chasm. Claude Code is the delivery vehicle. Its trajectory — surpassing \$2.5B in annualized revenue [12] and doubling since January — indicates a large and growing base. Exact user counts are not public; a rough revenue/ARPU inference suggests 2–5M Claude Code users as of Q1 2026 (assuming a blended ARPU of \$40–100/month across Pro, Max 5x, Max 20x, and API users), though this estimate carries significant uncertainty. Claude Code is increasingly adopted by non-programmers [5], expanding the potential audience beyond the technical core. The trajectory matters more than the current number: if Claude Code doubles again by year-end, the denominator is 4–10M.

Distribution nuance. Claude Code is the right *architecture* for this tool: a terminal-based agent that can run structured conversations, generate LaTeX, compile PDFs, and iterate on documents. But Claude Code is imperfect *distribution* for some segments. Segment 1 users (Lovable/Bolt/Replit builders) may never install Claude Code — they build in browser-based tools, not terminals. For them, prfaq is reachable only if Claude Code’s distribution expands to meet them (e.g., via a web interface or IDE integration) or if they migrate to Claude Code as their projects grow. Segment 2 users (expert PMs) and technical founders are a natural fit — they already live in terminals or are comfortable adopting developer tools. The initial adoption will come from where architecture and distribution overlap: founders, engineers, and savvy PMs who already use Claude Code. The broader segments represent trajectory upside, not launch-day TAM.

Beachhead: Amazon alumni and Working Backwards practitioners. Within Segment 2, a distinct micro-segment offers the highest early-adoption conviction: engineers and leaders who practiced Working Backwards at Amazon and now work elsewhere. These practitioners already believe in the PR/FAQ process — they do not need education on the framework or persuasion of its value. Their gap is operational: they lack the institutional support (templates, trained reviewers, organizational muscle memory) that made the process frictionless inside Amazon.

The plugin fills that gap. This micro-segment is small relative to the total addressable audience, but it is the most efficient beachhead: zero education cost, high framework fidelity expectations (which sharpens the product), and natural word-of-mouth to colleagues curious about the process. The author's professional network provides direct access to this group.

The meeting feature changes the value proposition for each segment. The review meeting commands (/prfaq:meeting and /prfaq:meeting-hive) shift what the plugin *is*. For Segment 1, the meeting is the most transformative feature. Most non-technical builders have never sat in a product review meeting. They have never had a principal engineer question their scaling assumptions, a unit economics reviewer challenge their pricing model, or a UX bar raiser ask how a first-time user would react. The meeting gives them an experience previously available only inside companies with mature product organizations. It is the difference between a tool that helps you write a document and a tool that helps you *think like a product team*. For Segment 2, the meeting is a force multiplier. An experienced PM knows what a review meeting should surface — but running one requires assembling five busy people in a room, often the bottleneck. The autonomous hive variant lets a solo PM get cross-functional scrutiny in minutes, not weeks of calendar negotiation. For Segment 3, the meeting is the feature most likely to drive adoption and word-of-mouth: it is visceral, surprising, and difficult to replicate outside a structured multi-agent system.

This is not a TAM-based revenue argument (the plugin is free); it is a reach argument. The question is how many builders the plugin can help, not how many it can monetize. Current addressable reach via Claude Code: 2–5M users (see above for methodology and caveats), of which the product-decision-making subset (founders, technical leads, PMs) is 10–25%, yielding 200K–1.25M reachable users today. The trajectory of both Claude Code adoption and the broader vibe-coding wave suggests this range grows significantly over 12–18 months.

Q11: What evidence do we have that customers want this?

Three categories of indirect evidence plus one anecdotal demand signal, but no primary customer data yet. First, the “vibe coding” phenomenon: thousands of products are built at speed without product discipline, and rebuilds are visible across startup communities and forums. Second, the explosion of “how to think about product” content targeting builders — books, courses, and newsletters suggest unmet demand for product skills among people who build with AI tools but lack formal product training. Third, the Working Backwards process has two decades of validation at Amazon [11], where it is mandatory for new product proposals. The gap is not whether the framework works — it is accessibility. Builders do not attend PM workshops. They install plugins.

Fourth, an organic demand signal from the author's professional network: a former colleague who learned the Working Backwards process from the author is introducing it at a new organization — and needs a lightweight way to practice the framework without the institutional support structure Amazon provides. This is the gap the plugin addresses: someone who values the process but lacks organizational scaffolding to run it. The plugin was partly inspired by this pattern: practitioners who leave environments where Working Backwards is institutionalized and want to carry the discipline with them. One anecdote is not validation, but it is a concrete instance of the target user and problem occurring organically.

Note: we have not validated that technical builders want product frameworks delivered inside Claude Code. Five to ten customer discovery interviews with engineers and technical founders would significantly strengthen this evidence base. The author's professional network of Amazon alumni and Working Backwards practitioners provides a high-conviction recruitment pool — people who already understand

and value the framework and can evaluate whether this delivery mechanism adds real value. As the non-technical audience grows [5], a separate round of interviews with that segment would inform whether `/prfaq:import` resonates.

Q12: What is your next step to validate your vision?

Ship the plugin publicly and recruit 10 target users — engineers and technical founders who use Claude Code — to run `/prfaq` on a real product idea and report what happened. This is the most important action: put the tool in front of real builders and observe whether they finish a document, whether they share it, and whether it changes their decision about what to build. Everything else (metrics, structured feedback, meeting feature testing) follows from having real users with real opinions. The detailed validation plan is in [FAQ 13](#).

Q13: How will we validate that builders actually want this?

The customer evidence FAQ (FAQ 11) is honest: we have strong indirect evidence but no primary customer data. The validation plan is straightforward.

Phase 1: Hands-on trials (weeks 1–4). Ask 10–20 target users to install the plugin and use it on a real product idea. Recruit deliberately across the spectrum: (1) casual builders and vibe coders from the punt-labs network and non-technical Claude Code communities, (2) engineers and technical founders from Claude Code channels (Discord, Reddit r/ClaudeAI) and Hacker News “Show HN” threads, (3) experienced product managers or product-minded founders who use structured frameworks, and (4) Amazon alumni and Working Backwards practitioners from the author’s professional network — the highest-conviction recruits because they already value the framework and can evaluate whether the plugin faithfully delivers it. This fourth group is uniquely valuable for early feedback: they will judge the plugin against their lived experience with the process, not an abstract expectation. This segmented recruitment is essential to test for the positioning gap — whether the plugin is too rigorous for casual builders and too untrusted for professionals — while the alumni cohort provides a baseline of users who need no persuasion on the framework’s value. The ask: try `/prfaq` on your next idea, run the full workflow through `/prfaq:meeting` or `/prfaq:meeting-hive`, and report what happened. The trial must cover the meeting feature explicitly — it is the primary differentiator and the feature with the most uncertainty about whether users find simulated cross-functional debate genuinely useful versus theatrical.

Phase 2: Qualitative feedback (concurrent with trials). The plugin ships a feedback command (`/prfaq:feedback-to-us`) — a 1–5 satisfaction rating with an optional comment. This is the first feedback signal. During the trial, we ask participants to rate after each completed document. No document content is transmitted; the plugin’s default is zero telemetry.

Phase 3: Structured feedback channel (exploratory). If the basic 1–5 rating proves too shallow, we will expand the feedback command to capture structured responses — what worked, what did not, and whether the output was useful enough to share — via a single API call. This reduces friction of collecting qualitative data from terminal-native users unlikely to fill out a Google Form. The command would remain opt-in, clearly labeled, and would transmit only the user’s explicit responses. This is exploratory; we build it only if Phase 1 reveals users want to give richer feedback than a 1–5 rating.

Signals we are looking for:

1. **Completion rate** — do users finish a full PR/FAQ, or abandon mid-session? Target: 60%+ completion among trial participants.

2. **Shareability** — do users share the generated PDF with a co-founder, investor, or advisor? Target: 30%+ of completed documents are shared.
3. **Decision impact** — did the PR/FAQ change the user's decision about what to build? Even "I decided not to build it" is a strong positive signal.
4. **Repeat usage** — do users run /prfaq on a second idea? Repeat usage within 30 days is the strongest signal of product-market fit for a free tool.
5. **Unprompted referral** — do trial users recommend the plugin to others without being asked?
6. **Meeting utility** — do users who run /prfaq:meeting or /prfaq:meeting-hive report that the simulated debate surfaced a blind spot they had not considered? Do they change the document as a result? Target: 50%+ of meeting users identify at least one substantive insight they would not have found alone. This is the critical signal for the meeting feature's value proposition.
7. **Positioning gap** — do casual builders (vibe coders, non-technical founders) abandon the process because it feels too rigorous, while experienced PMs dismiss the output because they do not trust AI-generated strategic analysis? The trial must recruit from both ends of the spectrum, not just the technical middle. If both segments reject the plugin for opposite reasons — too heavyweight for one, too untrusted for the other — the product has a positioning problem that no amount of iteration can fix without a fundamental scope or audience decision.

Timeline: Recruit the first 10 trial users within 2 weeks of public launch. Complete the 10–20 person trial within 6 weeks. Synthesize findings into a revised customer evidence base. If fewer than 40% of trial users complete a PR/FAQ, or if qualitative feedback consistently indicates the output is not useful enough to share, upgrade the value risk rating to High and reconsider the product strategy.

Q14: Who are the competitors and why will we win?

The most credible alternative is pasting a PR/FAQ template into Claude.ai. What it cannot replicate: a multi-persona review meeting where four experts with distinct risk lenses debate your document and surface blind spots. You cannot simulate that by pasting a template into a chat. The meeting commands (/prfaq:meeting and /prfaq:meeting-hive) are the widest moat — they require orchestrated multi-agent architecture, calibrated persona prompts, and a structured debate protocol a single-turn chat cannot approximate. Beyond the meeting, the template-in-chat approach lacks: structured reference guides that catch common mistakes, the four risks evaluation framework, automated peer review against cognitive biases, LaTeX compilation to a shareable PDF, and a repeatable process that improves across uses. The plugin encodes twenty years of Working Backwards practice into a reusable workflow; a blank chat starts from zero every time.

Traditional competitors include product management tools (Productboard, Aha!, Notion templates) and business planning tools (Lean Canvas generators). None are integrated into the builder's working environment. They require context-switching: leave the current tool, open a browser, fill in a form. A Claude Code plugin runs in the same session where the builder already works — the shift from "building" to "product thinking" happens in a single command, not a tab switch. None offer a simulated review meeting.

The most credible competitive response is Anthropic itself. First, Anthropic could feature the plugin — promote it in their plugin directory, recommend it in onboarding, or highlight it as a case study. If that happens, we get distribution we cannot buy. Second, Anthropic could build a

native product-thinking feature into Claude Code. If that happens, they validate the category: the idea that terminal-based AI tools should help users decide *what* to build, not just *how* to build it. Both outcomes are positive for the product-thinking-in-terminal thesis. The strategic exposure is real — Anthropic controls the plugin system, the distribution, and the competitive landscape — but the downside scenarios carry embedded upside.

Our depth advantage persists in either scenario. The plugin encodes Amazon’s Working Backwards process specifically [11], with reference guides, a peer review framework grounded in Cagan’s four risks [4], and a Kahneman-informed decision quality checklist. A general-purpose product-thinking feature is unlikely to replicate that specificity. Users who value a framework with two decades of track record at Amazon would continue to prefer the specialized tool even if a native alternative exists. The mitigation against platform concentration is explored in the Feature Appendix: alternative CLI-based distribution channels reduce single-platform dependency over time.

Technical

Q15: What are the major technical risks?

The full V1 feature set is shipped, including multi-agent review meetings (/prfaq:meeting in v0.6.0, /prfaq:meeting-hive in v0.8.0). The plugin is a set of markdown files plus a LaTeX template and a shell script. No backend, no API, no database. The LaTeX dependency requires a TeX distribution installed, a mild friction point; the installer mitigates with platform-specific setup instructions.

Multi-agent orchestration — the primary technical challenge — is solved and shipping. The remaining refinement risk is persona fidelity across Claude model updates: each agent must stay in character, disagree substantively (not performatively), and produce debate that is genuinely useful rather than theatrical. If a model update degrades persona quality, the meeting commands could regress from “simulated review meeting” to “five versions of the same generic feedback.” Mitigation: prompt regression testing after model updates and persona calibration from user feedback.

The deeper risk is strategic, not technical. The plugin depends on Claude Code’s plugin system for loading, execution, and distribution. Anthropic controls all three. If Anthropic deprecates the plugin API, restricts third-party plugins, or ships a native alternative, the migration cost is real: prompt engineering must be adapted to a new host, distribution rebuilt from scratch, and the installed base may not follow. The plugin’s architecture — markdown files, a LaTeX template, a shell script — is portable in principle, but retargeting to another CLI-based agent (Codex CLI, Gemini CLI) requires non-trivial adaptation. This platform dependency is why feasibility risk is rated Medium rather than Low.

Q16: What dependencies exist on other teams or systems?

The plugin depends on Anthropic’s Claude Code plugin system for loading and execution, and on a local TeX distribution for PDF compilation. Both are external dependencies outside our control. The Claude Code plugin API is stable (v1), and TeX Live is a mature, 30-year-old ecosystem. If Anthropic changes the plugin format, migration is straightforward — the plugin is a handful of files with simple structure. If TeX Live is unavailable, the plugin still generates the .tex source file; only PDF compilation is affected.

All output is portable. The .tex source files and compiled PDFs are standalone artifacts — they do not require the plugin to read, edit, or compile. If the plugin becomes unavailable, users

retain their documents and can compile them with any standard TeX distribution. No content is locked in a proprietary format or dependent on a running service. This portability is deliberate: the plugin is a workflow tool, not a platform, and the documents it produces belong to the user.

Q17: If this succeeds, what breaks first?

Nothing at the infrastructure level — the plugin is stateless, runs locally, and requires no servers. At scale, the pressures are human, not technical: (1) GitHub issue volume — at 1,000+ users, support requests and feature demands will exceed solo maintainer capacity; mitigation is clear contribution guidelines and aggressive scope discipline (see Won't Do list); (2) prompt maintenance — as Claude models evolve, skill prompts may need tuning to maintain output quality; (3) backwards compatibility — if Anthropic changes the plugin system, the plugin must migrate; the simple file-based architecture makes this low-risk; (4) community expectations — a popular free tool attracts requests for features that contradict the design philosophy (dashboards, collaboration, web interfaces). The Won't Do list makes these decisions in advance, not under pressure.

Q18: What is the estimated development timeline?

The full V1 feature set through v0.8.1 is built and shipping. This includes `/prfaq:meeting`, which shipped in v0.6.0 with four agentic personas and visible debate threading, and its autonomous variant `/prfaq:meeting-hive`, which shipped in v0.8.0 via `claude-flow` multi-agent orchestration. The core workflow (`/prfaq`, `/prfaq:feedback`, `/prfaq:review`, `/prfaq:research`, `/prfaq:feedback-to-us`) and supporting infrastructure (stage awareness, version tracking, cross-references, reference guides) are shipped. Remaining planned work is `/prfaq:import`, which requires document parsing across multiple input formats (plain text, Markdown, Google Doc paste, raw `.tex`) and gap analysis logic — estimated at one to two development sessions. Other planned features (Markdown output mode, comparative analysis, template customization, alternative CLI distribution) are lower priority and carry no technical risk. No infrastructure to build — remaining complexity is in prompt engineering and skill orchestration. Refinement risks are limited to prompt tuning as Claude models evolve and maintaining persona fidelity across model updates in the meeting commands.

Business

Q19: What is the revenue model?

None. `prfaq` is free and open source under a permissive license. The strategic value is threefold: (1) it establishes `punt-labs` as the reference implementation for AI-assisted product development tooling — the plugin that defines how structured product thinking works inside an AI coding agent; (2) it builds reputation and visibility in the Claude Code ecosystem; (3) it creates a feedback loop for understanding how builders approach product thinking, which informs other products at `punt-labs`.

Q20: Is there a commercial model?

Not today, and not without evidence. The plugin is free and will remain free. However, if product-market fit is confirmed through the validation plan (see [FAQ 13](#)), there is a natural premium tier: subscription access to professional research sources. The free plugin uses web search to find evidence for claims. A premium research tier could provide access to professional

databases — Pitchbook for market sizing, Statista for industry statistics, proprietary industry reports — as well as AI-powered survey agents that conduct lightweight customer discovery interviews on the user's behalf. These capabilities address the weakest link in most PR/FAQs: the evidence base. A builder who can generate a PR/FAQ with real market data and structured customer signals has a meaningfully better decision document than one relying on free web search.

This is a path, not a plan. The decision to build a commercial tier is gated on three conditions: (1) the free plugin achieves PMF — repeat usage, shareability, and community traction as defined in [FAQ 13](#); (2) user feedback identifies research quality as a bottleneck; (3) the unit economics of licensing professional data sources close at a price point the target audience will pay. Until all three conditions are met, the commercial model remains a documented option, not a roadmap item.

Q21: What does the cost structure look like at steady state?

prfaq has zero infrastructure cost — no servers, no databases, no APIs. The real cost is maintainer time. At current scale (pre-launch): 2 hours per week on development and prompt refinement. At the 500-star target: 4–8 hours per week covering issue triage, community engagement, prompt maintenance as Claude models evolve, and occasional LaTeX template updates. At 1,000+ users: maintainer capacity becomes the constraint. The plugin is designed to be self-sustaining: the entire codebase is markdown files, a LaTeX template, and a shell script — no servers, no API keys, no databases, no backend. Install once, use forever. If punt-labs disappears tomorrow, the plugin continues working. This zero-infrastructure architecture makes it trivially forkable. If maintainer capacity is exceeded, the community can maintain it independently. At that scale, the project either recruits community co-maintainers or narrows support scope to bug fixes only. The opportunity cost is hours not spent on other punt-labs projects. This is acceptable while the plugin establishes punt-labs as the reference implementation for AI-assisted product development tooling; re-evaluate if other punt-labs priorities require the same hours.

Q22: What are the key metrics for success?

Five metrics, measured via GitHub analytics, installer telemetry, plugin feedback, and community signals:

1. **Install count** — target: 500 installs within 6 months. Measured via GitHub release download counts and install script fetch logs.
2. **User satisfaction ratings** — target: average 4.0+ out of 5.0 from users who run the feedback command. Shipped in v0.7.0; captures a 1–5 score with an optional comment.
3. **GitHub stars and forks** — target: 500 stars within 6 months. Measures discoverability and perceived value.
4. **Issues and pull requests from external contributors** — target: 10 external PRs within 6 months. Measures whether the plugin is useful enough that people invest time improving it.
5. **Mentions in builder and developer communities** (Hacker News, Twitter/X, Reddit, Discord servers) — target: 3 organic mentions within 3 months. Measures whether the plugin solves a real problem people tell others about.

If none of these targets are met after 6 months, the plugin is not reaching its audience and the distribution strategy should be reconsidered.

Q23: Why now? What has changed?

Three shifts converged. First, AI coding tools reached mainstream adoption in 2025–2026, creating a large population of engineers and technical founders who can build products independently but lack product discipline. Second, Claude Code’s plugin system matured, making it possible to deliver structured workflows inside the developer’s existing tool. Third, the consequences of “vibe coding without product thinking” are becoming visible — failed launches, rebuilds, wasted months — creating demand for lightweight product frameworks that do not require hiring a PM or leaving the terminal. A fourth tailwind is emerging: Claude Code is crossing over to non-technical users [5], expanding the potential audience beyond the technical core.

Q24: What is the customer acquisition strategy?

Distribution is the product strategy for a free plugin. Primary channels: (1) GitHub discoverability — README, topics, and stars drive organic search traffic; (2) community seeding — posts on Hacker News, Reddit r/ClaudeAI, and Twitter/X targeting the “vibe coding” and AI-builder audience; (3) content marketing — the plugin’s own PR/FAQ serves as a case study and demo artifact; (4) word-of-mouth — builders who find the plugin useful share the PDF output, which includes a link back to the repository. No paid acquisition.

Highest-conviction channel: Amazon alumni network. The author’s professional network includes direct access to Amazon alumni — engineers and leaders who already understand and value the Working Backwards process. This is the most efficient early adopter pool: these practitioners do not need to be sold on the framework, only on the delivery mechanism. They already know what a PR/FAQ is, they already believe in the process, and many now work at companies that lack the institutional support to run it. The plugin solves their specific problem: practicing Working Backwards without organizational scaffolding. Outreach to this network is the lowest-cost, highest-conversion acquisition channel at launch.

Indirect channel: Working Backwards consulting ecosystem. The author has a direct professional relationship with the author of the Working Backwards methodology [11], who maintains an active consulting practice helping enterprise clients adopt the process. This relationship provides two forms of leverage: (1) direct feedback on the plugin from someone who wrote the book on the framework, and (2) indirect distribution through the consulting practice’s client base — organizations actively learning Working Backwards and may benefit from a lightweight tool that reinforces the discipline. This is not a formal distribution partnership; it is a professional relationship that creates a natural feedback and awareness channel.

The secondary audience — non-technical Claude Code users [5] — is not actively targeted in V1. They discover the plugin organically through Claude Code’s plugin ecosystem and through technical users who share the PDF output. This is spillover, not acquisition strategy.

Estimated time investment: 2–4 hours per week on community engagement for the first 3 months. Conversion assumption: a front-page Hacker News post reaches 20–30K views and converts at 0.1–0.2%, yielding 20–60 installs per post. The Amazon alumni network is expected to yield higher conversion rates — these are warm contacts who already value the framework — but the exact numbers are testable hypotheses, not forecasts. If organic channels do not work within 6 months, the distribution hypothesis is wrong and should be revisited.

Q25: What are we not building?

Not a SaaS product management platform. Not a competitor to Jira, Linear, Productboard, or Notion. No collaboration features, dashboards, or analytics. No web interface. The plugin is a single-player tool that runs locally, produces a document, and gets out of the way. Scope discipline is the feature. See the Feature Appendix for the full Shipped / Planned / Won't Do breakdown.

Q26: It is one year from now and prfaq failed. What went wrong?

Most likely failure scenario: builders do not want product discipline, even when free and frictionless. The vibe coding audience builds for the joy of building, and a tool that asks “what evidence do you have that customers want this?” feels like a buzzkill, not a feature. Second scenario: the plugin does not surface in a crowded marketplace — the Claude Code plugin ecosystem is growing rapidly, and discoverability is poor. If users cannot find `prfaq` among hundreds of alternatives, organic adoption stalls regardless of quality. Third scenario: the output quality is not high enough to share with stakeholders — if the generated PR/FAQ reads like AI slop rather than a credible decision document, users will not trust it for real decisions. Fourth scenario: the PR/FAQ process is too heavyweight for the target audience — builders want a quick sanity check, not a formal document, and a lighter-weight alternative captures the market. Fifth scenario: maintenance burden — the shipped meeting commands (`/prfaq:meeting` and `/prfaq:meeting-hive`) require ongoing persona calibration across Claude model updates, consuming maintainer time that should go toward community engagement and adoption. Sixth scenario: we misidentified the customer or their motivation — the tool solves the wrong person's problem, or targets the right person for the wrong reason. Builders may want stakeholder theater (a polished-looking document that signals rigor) rather than genuine product discipline, and when `prfaq` forces real answers to hard questions, they abandon it for something that produces impressive artifacts without uncomfortable introspection. Seventh scenario: the product falls between two stools — too rigorous for vibe coders, too untrusted for professionals. Casual builders who thrive on intuition and speed find the structured PR/FAQ process heavyweight and prescriptive: it demands customer definitions, competitive analysis, and unit economics from people who just want to ship. Meanwhile, experienced product managers and executives do not trust AI-generated strategic analysis for real decisions — they view the output as a parlor trick, not a credible basis for resource allocation. The result is a positioning gap where neither audience fully adopts the tool: the casual segment bounces off the rigor, the professional segment bounces off the provenance. This is distinct from scenarios 1 and 4, which describe resistance from a single audience; the risk here is that the plugin's positioning alienates *both* ends of the spectrum simultaneously, leaving no natural home.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Market-level evidence strong (proven framework, large technical audience). Customer-level evidence absent — no interviews confirming demand for this delivery mechanism from engineers and technical founders. Positioning risk: the plugin may be too rigorous for casual builders (who want speed, not discipline) and too untrusted for professional PMs (who do not rely on AI-generated strategic analysis for real decisions) — falling between two stools with no natural home in either segment. Plugin is free, so adoption barrier is low. Mitigation: 10–20 person validation trial with opt-in metrics and structured feedback (see FAQ 13), recruiting from both casual and professional segments to test for positioning gap.
Usability	Low	One-line install, one slash command, Claude guides interactively. TeX dependency is the main friction; installer mitigates. Time to value under one hour.
Feasibility	Medium	Full V1 feature set shipped through v0.8.1, including multi-agent meeting simulation. No backend, no infrastructure. Remaining planned work (/prfaq:import, Markdown output, comparative analysis) is straightforward. Refinement risks limited to prompt tuning across Claude model updates and maintaining persona fidelity in the meeting commands. Strategic risk is platform dependency: the plugin runs exclusively on Claude Code, and Anthropic controls the plugin system, distribution, and competitive response. If Anthropic deprecates the plugin API or ships a native alternative, migration cost is non-trivial. Mitigation: portable LaTeX output (users retain documents regardless of plugin availability), exploration of alternative CLI-based distribution channels, and architecture that minimizes platform-specific coupling.
Viability	Medium	Zero infrastructure cost but maintainer time is the binding constraint. At 500+ stars, estimated 4–8 hrs/week on issues, community, and prompt maintenance. The plugin is designed to survive maintainer abandonment: no servers, no API keys, no databases — just markdown files, a LaTeX template, and a shell script, plus standalone LaTeX output. Engineer-heavy user base naturally tracks potential co-maintainers. Mitigation: contribution guidelines, scope discipline, forkable architecture.

Feature Appendix

Shipped (V1)

- F1. Interactive discovery workflow** — structured questions that guide the user from idea to document. Shipped v0.1.0.
- F2. Complete PR/FAQ document generation** — press release, external FAQs, internal FAQs, four risks assessment. Shipped v0.1.0.
- F3. LaTeX compilation to professional PDF** — shareable artifact, not a disposable brainstorm. Shipped v0.1.0.
- F4. Reference guides** — encoded best practices for PR structure, FAQ structure, four risks, common mistakes, unit economics, feasibility, and usability. Shipped v0.1.0.
- F5. Peer review agent (`/prfaq:review`)** — automated critique against Working Backwards principles and cognitive bias checklist. Shipped v0.2.0.
- F6. Research sub-agent (`/prfaq:research`)** — scans local `./research/` directory, indexed documents, and the web for evidence. Shipped v0.3.0.
- F7. Biblatex citation system** — academic-style citations for claims, linking evidence to assertions. Shipped v0.3.0.
- F8. `/prfaq:feedback`** — directed iteration from pointed user feedback; traces cascading effects via section dependency graph and surgically redrafts affected sections. Shipped v0.4.0.
- F9. `/prfaq:feedback-to-us`** — anonymous satisfaction feedback — 1–5 rating with optional comment to help improve the plugin. No document content transmitted. Shipped v0.7.0.
- F10. `/prfaq:meeting`** — simulates an Amazon-style PR/FAQ review meeting with four agentic personas (target customer, principal engineer, unit economics reviewer, UX bar raiser) plus a skeptical executive. Agent debate is visible to the user. Main Claude facilitates and synthesizes debate into pointed decision questions. The user is the PM and final decision-maker. Decisions feed into `/prfaq:feedback` for automated iteration. Shipped v0.6.0.
- F11. `/prfaq:meeting-hive`** — orchestrates the review meeting as autonomous multi-agent debate via claude-flow. Personas reach consensus without the user moderating every exchange, surfacing conflicts and agreements independently. Shipped v0.8.0.
- F12. Stage awareness** — document stage (hypothesis, validated, growth) calibrates evidence expectations across all commands. Shipped v0.5.0.
- F13. Version tracking** — major/minor versioning with automatic increment after feedback cycles. Shipped v0.5.0.
- F14. Cross-references** — clickable `\faqref` and `\featureref` links between the press release, FAQs, and Feature Appendix. Shipped v0.4.0.
- F15. `/prfaq:streamline`** — scalpel editor that removes redundancy across sections, eliminates weasel words and hollow adjectives, compresses inflated phrases, and applies the “so what” test. Targets 10–20% length reduction without touching evidence, citations, or risk assessments. Shipped v0.8.1.

Planned

- F16. `/prfaq:import`** — ingest an existing PR/FAQ draft (plain text, Markdown, or raw `.tex`), parse and structure it against the framework, research evidence for unsupported claims, format into the LaTeX template with proper environments and citations, compile to PDF, present a gap analysis
- F17. Markdown output mode** — alternative to LaTeX for users without a TeX distribution

- F18. Comparative analysis mode** — evaluate two competing product ideas side-by-side
- F19. Template customization** — allow users to override the LaTeX template for branding
- F20. Alternative CLI-based distribution channels** — explore Codex CLI, Gemini CLI, and other terminal-based AI coding tools as additional distribution surfaces. The plugin's architecture — markdown skill files, a LaTeX template, and a shell script — is not deeply coupled to Claude Code's plugin system. Porting to other CLI-based agents would reduce single-platform concentration risk and expand reach to builders who use different tools. Not about supporting multiple LLM backends simultaneously; about ensuring the product-thinking workflow is available wherever builders work in terminals. Priority increases if Claude Code's plugin ecosystem becomes restrictive or if a competing CLI tool gains significant share among the target audience.

Won't Do

- F21. Web dashboard or SaaS platform** — the plugin runs locally inside Claude Code. Adding a web layer would introduce infrastructure, authentication, and hosting costs that contradict zero-ops design. If users want to share documents, they share the PDF.
- F22. Collaboration or multi-user editing** — PR/FAQ is a single-author process by design. The document is written by one person, then reviewed by others. Real-time collaboration would encourage design-by-committee, which Working Backwards explicitly avoids.
- F23. Project management features** — no task tracking, no roadmap views, no sprint planning. The plugin produces a decision document, not a project plan. Users who need project management have Linear, Jira, or beads.

References

- [1] Stack Overflow. *2025 Developer Survey*. Annual survey of 65,000+ developers on tools, practices, and AI adoption. 2025. URL: <https://survey.stackoverflow.co/2025>.
- [2] Andrej Karpathy. *Vibe Coding*. Original post coining the term “vibe coding” — viewed over 4.5 million times. Feb. 2025. URL: <https://x.com/karpathy/status/1886192184808149383>.
- [3] HFS Research and Publicis Sapient. *Smash Through Tech Debt: Why AI Is the Jackhammer*. Survey of 608 IT and business leaders estimating Global 2000 accumulated technical debt at \$1.5–2 trillion. 2025. URL: <https://www.hfsresearch.com/research/publicis-sapient-jackhammer/>.
- [4] Marty Cagan. *Inspired: How to Create Tech Products Customers Love*. 2nd ed. Wiley, 2018. ISBN: 978-1119387503.
- [5] Natallie Rocha. “This A.I. Tool Is Going Viral. Five Ways People Are Using It.” In: *The New York Times* (Jan. 2026). Profiles five non-programmers using Claude Code: school administrator, photographer, prosecutor, finance professor, welding business owner. URL: <https://www.nytimes.com/2026/01/23/technology/claude-code.html>.
- [6] TechCrunch. *Lovable Says It’s Nearing 8 Million Users*. Lovable, an AI coding startup founded in 2024, nearing 8 million users within one year. Nov. 2025. URL: <https://techcrunch.com/2025/11/10/lovable-says-its-nearing-8-million-users-as-the-year-old-ai-coding-startup-eyes-more-corporate-employees/>.
- [7] Amjad Masad. *Replit CEO: We Don’t Care About Professional Coders Anymore*. Replit’s 35M+ users; CEO explicitly pivoting away from professional developers toward non-coders. Jan. 2025. URL: <https://www.analyticsvidhya.com/blog/2025/01/replit-ceo-we-dont-care-about-professional-coders-anymore/>.
- [8] Sacra. *Bolt.new Revenue, Funding & News*. Bolt.new hit \$40M ARR in March 2025, progressing from \$4M ARR within 4 weeks of launch. 2025. URL: <https://sacra.com/c/bolt-new/>.
- [9] Sacra. *Cursor Revenue, Valuation & Funding*. Cursor reached \$1B ARR and 1M daily active users. 2025. URL: <https://sacra.com/c/cursor/>.
- [10] TechCrunch. *A Quarter of Startups in YC’s Current Cohort Have Codebases Almost Entirely AI-Generated*. 25% of Y Combinator W25 cohort startups have codebases that are almost entirely AI-generated. Mar. 2025. URL: <https://techcrunch.com/2025/03/06/a-quarter-of-startups-in-ycs-current-cohort-have-codebases-that-are-almost-entirely-ai-generated/>.
- [11] Colin Bryar and Bill Carr. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin’s Press, 2021. ISBN: 978-1250267597.
- [12] Constellation Research. *Anthropic’s Claude Code Revenue Doubled Since Jan. 1*. Claude Code annualized run-rate revenue reached approximately \$2.5B as of February 2026, more than doubling since January 1. Feb. 2026. URL: <https://www.constellationr.com/insights/news/anthropics-claude-code-revenue-doubled-jan-1>.